



Programming Java

Interfaces & Packages

Incheon Paik



Contents

- Interfaces
- Interface References
- Interface Inheritance
- The instanceof Operator
- Packages
- Classpath
- The import Statement
- Access Control and Packages



Interfaces

An Interface Declaration

```
intfModifier interface intfName {
    varModifier1 type1 varName1 = value1;
    varModifier2 type2 varName2 = value2;
    varModifier3 type3 varName3 = value3;
    .....
    varModifierN typeN varNameN = valueN;
    mthModifier1 rtype1 mthName1(params1);
    mthModifier2 rtype2 mthName2(params2);
    .....
    mthModifierN rtypeN mthNameN(paramsN);
}
```

Implements Keyword

```
clsModifiers class clsName extends superName implements intfList
{
    // implementation
    .....
}
```

```
interface Shape2D {
    double getArea();
}

interface Shape3D {
    double getVolume();
}

class Point3D {
    double x, y, z;

    Point3D(double x, double y, double z) {
        this.x = x;
        this.y = y;
        this.z = z;
    }
}
```



Interfaces

```
abstract class Shape {
    abstract void display();
}

class Circle extends Shape
implements Shape2D {
    Point3D center, p; // p is an point on circle

    Circle(Point3D center, Point3D p) {
        this.center = center;
        this.p = p;
    }

    public void display() {
        System.out.println("Circle");
    }

    public double getArea() {
        double dx = center.x - p.x;
        double dy = center.y - p.y;
        double d = dx * dx + dy * dy;
        double radius = Math.sqrt(d);
        return Math.PI * radius * radius;
    }
}
```

Result :

```
Circle
3.141592653589793
Sphere
4.1887902047863905
```

```
class Sphere extends Shape
implements Shape3D {
    Point3D center;
    double radius;

    Sphere(Point3D center, double radius) {
        this.center = center;
        this.radius = radius;
    }

    public void display() {
        System.out.println("Sphere");
    }

    public double getVolume() {
        return 4 * Math.PI * radius * radius * radius / 3;
    }
}

class Shapes {
    public static void main(String args[]) {
        Circle c = new Circle(new Point3D(0, 0, 0), new
        Point3D(1, 0, 0));
        c.display();
        System.out.println(c.getArea());
        Sphere s = new Sphere(new Point3D(0, 0, 0), 1);
        s.display();
        System.out.println(s.getVolume());
    }
}
```

Interfaces

```
interface Material {
    String bronze = "bronze";
    String gold = "gold";
    String marble = "marble";
    String silver = "silver";
    String wood = "wood";
}

abstract class MaterialObject {
    String material;
}

class Ball extends MaterialObject {
    Ball(String material) {
        this.material = material;
    }
}

class Coin extends MaterialObject {
    Coin(String material) {
        this.material = material;
    }
}
```

```
class Ring extends MaterialObject {
    Ring(String material) {
        this.material = material;
    }
}

class MaterialObjects {
    public static void main(String args[]) {
        Ball ball = new Ball(Material.wood);
        Coin coin = new Coin(Material.silver);
        Ring ring = new Ring(Material.gold);
        System.out.println(ball.material);
        System.out.println(coin.material);
        System.out.println(ring.material);
    }
}
```

Result :

```
wood
silver
gold
```

Interface References

Referencing an Interface Variable & Interface Variable

```
intfRef.varName
intfRef.mthName(args)
```

```
interface A {
    void display(String s);
}

class C1 implements A {
    public void display(String s) {
        System.out.println("C1: " + s);
    }
}

class C2 implements A {
    public void display(String s) {
        System.out.println("C2: " + s);
    }
}

class C3 implements A {
    public void display(String s) {
        System.out.println("C3: " + s);
    }
}
```

```
class InterfaceReferenceVariable {
    public static void main(String args[]) {
        A a;
        a = new C1();
        a.display("String 1");
        a = new C2();
        a.display("String 2");
        a = new C3();
        a.display("String 3");
    }
}
```

Result :

```
C1: String 1
C2: String 2
C3: String 3
```



Interface Inheritance

Extending Interface

```
intModifier interface intfname extends intfList {
    // interface body
}
```

```
interface J {
    int j = 200;
    int j1();
}

interface K {
    double k1();
}

interface L extends J, K {
    boolean l1();
}

class I implements L {
    public int j1() {
        return 4;
    }

    public double k1() {
        return 6.8;
    }

    public boolean l1() {
        return true;
    }
}
```

```
class InterfaceInheritance {

    public static void main(String args[]) {
        I i = new I();
        System.out.println(i.j);
        System.out.println(i.j1());
        System.out.println(i.k1());
        System.out.println(i.l1());
    }
}
```

Result :

200
4
6.8
true



Interface Inheritance

```
interface Base {
    int base = 0;
    int ambiguous = 1000;
}

interface Set1 extends Base {
    int set1 = 1;
    int ambiguous = 1000;
}

interface Set2 extends Base {
    int set2 = 2;
}

interface Total extends Set1, Set2 {
    int total = 3;
}
```

```
class Z implements Total {
}

class AmbiguousVariable {
    public static void main(String args[]) {
        Z z = new Z();
        System.out.println(z.base);
        System.out.println(z.set1);
        System.out.println(z.set2);
        System.out.println(z.total);
        System.out.println(z.ambiguous); // Error
    }
}
```

Fields in interface :

public, static, final



Interface Inheritance

```
interface L1 {
    void f();
    void g();
}

interface L2 extends L1 {
    void f();
    int g();
}

class CompileError {
    public static void main(String args[]) {
        System.out.println("Compile-time error");
    }
}
```

All methods in interface :
Implicitly abstract, public method

Result :
Compile Error



Defining the Generic Type

BinaryTree generic type

```
public class BinaryTree<T extends Comparable<T>> {

    public void add(T value) {
        // Add a value to the tree
    }

    // Create a list containing the values from the tree in sequence
    public LinkedList<T> sort() {
        // Code to extract object from the tree in sequence
        // and insert then in a LinkedList object and return that ....
    }

    LinkedList<T> values; // Stores sorted values
    private Node root; // The root node
}
```

```
// Private inner class defining nodes
private class Node {
    Node(T value) {
        obj = value;
        count = 1;
    }
    T obj; // Object stored in the node
    int count; // Count of identical nodes
    Node left; // The left child node
    Node right; // The right child node
}
```

The Comparable<> interface declares a single method, the compareTo() method. If you specify the Comparable<> interface as a constraint on the type parameter for the BinaryTree<> class, it ensures that all object added to a BinaryTree<> object implement the compareTo() method.

Defining the Generic Type

Inserting/Extracting Objects in a Binary Tree

Code Example:

<http://ebiz.u-aizu.ac.jp/~paikic/lecture/2005-1/code-examples/Java2-1.5/Code/Ch13/TryBinaryTree/BinaryTree.java>

Hidden Constraints in the BinaryTree<E> Type

The parameterized type has a built-in constraint that was not exposed by the examples storing String and Integer objects. Let's see a Person class example.

```
public class Person implements Comparable<Person> {
    public Person(String name) {
        this.name = name;
    }

    public int compareTo(Person person) {
        if (person == this) { return 0; }
        return this.name.compareTo(Person.name);
    }

    public String toString() { return name; }

    protected String name;
}
```

Java

11

Computer Industry Lab.

The instanceof Operator

The instanceof Operator

Varname instanceof type

Determine if an object is of a particular class or implements a specific interface.

For checking some object is an instance of specific class

Ex) if (point1 instanceof Circle)

```
abstract class Fish {
    abstract void display();
}

abstract class FreshWaterFish extends Fish {
}

abstract class SaltWaterFish extends Fish {
}

class Trout extends FreshWaterFish {
    void display() {
        System.out.println("Trout");
    }
}

class Flounder extends SaltWaterFish {
    void display() {
        System.out.println("Flounder");
    }
}
```

```
class Tuna extends SaltWaterFish {
    void display() {
        System.out.println("Tuna");
    }
}

class InstanceofClass {
    public final static int NUMFISH = 4;

    public static void main(String args[]) {

        // array of Fish
        Fish fishes[] = new Fish[NUMFISH];

        // Making of Object
        fishes[0] = new Trout();
        fishes[1] = new Flounder();
        fishes[2] = new Tuna();
        fishes[3] = new Trout();

        // Use instanceof
        for (int i = 0; i < NUMFISH; i++) {
            Fish fish = fishes[i];
            if (fish instanceof SaltWaterFish) {
                fish.display();
            }
        }
    }
}
```

Result :
Flounder
Tuna

Java

12

Computer Industry Lab.



The instanceof Operator

```
interface Vehicle {
    void drive();
}

abstract class Mammal {
}

class Bear extends Mammal {
}

class Elephant extends Mammal
implements Vehicle {
    public void drive() {
        System.out.println("Elephant: drive");
    }
}

class Horse extends Mammal
implements Vehicle {
    public void drive() {
        System.out.println("Horse: drive");
    }
}

class Lion extends Mammal {
}
```

```
class InstanceofInterface {
    public final static int NUMMAMMALS = 4;

    public static void main(String args[]) {

        // Array of mammals
        Mammal mammals[] = new
        Mammal[NUMMAMMALS];

        // Object
        mammals[0] = new Bear();
        mammals[1] = new Elephant();
        mammals[2] = new Horse();
        mammals[3] = new Lion();

        // Use instanceof
        for (int i = 0; i < NUMMAMMALS; i++) {
            if (mammals[i] instanceof Vehicle) {
                Vehicle v = (Vehicle)mammals[i];
                v.drive();
            }
        }
    }
}
```

Result :
Elephant: drive
Horse: drive



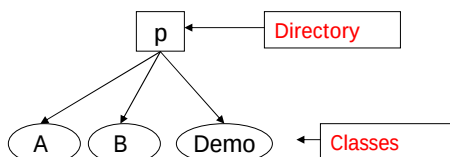
Packages

The package Statement

```
package packageName;
```

Packages of Java

```
java.applet
java.awt
java.awt.event
java.io
java.lang
java.net
java.util
```



```
package p;
class PackageDemo {
    public static void main(String args[]) {
        A a = new A();
        a.a1();
        B b = new B();
        b.b1();
        C c = new C();
        c.c1();
    }
}

package p;
class A {
    void a1() {
        System.out.println("a1");
    }
}

package p;
class B {
    void b1() {
        System.out.println("b1");
    }
}
```

Execution:

```
% javac p\*.java
% java p.PackageDemo
```

Access Control & Packages

Keywords

public : can be accessed from all classes

protected : In same package, and subclasses in other package

private : only in same class

```
package e;
public class E {
    public int e1 = 11;
    protected int e2 = 22;
    private int e3 = 33;
}

package f;
import e.*;

public class F extends E {
    public void display() {
        // can access for public in super
        System.out.println(e1);
        // can access for protected in super
        System.out.println(e2);
        // cannot access for private in super
        // System.out.println(e3);
    }
}
```

```
package g;
import e.*;

public class G {
    public void display() {
        // Instance of E
        E e = new E();

        // public member
        System.out.println(e.e1);

        // Cannot access protected
        // System.out.println(e.e2);

        // cannot access : private
        // System.out.println(e.e3);
    }
}

package h;
import f.F;
import g.G;
class ProtectedDemo {
    public static void main(String args[]) {
        F f = new F();
        f.display();

        G g = new G();
        g.display();
    }
}
```

Exercise

- Step 1, Step 2 : Refer to the examples in the exercise.

- Step 3 (Polymorphism)

```
class SuperClass {
    int value;
    SuperClass() {
        value = 0;
    }
    SuperClass(int i) {
        value = i;
    }
    void output() {
        System.out.println("SuperClass : " + value);
    }
}

class SubClass extends SuperClass {
    int value;
    SubClass (int i) {
        value = i;
    }
    void output() {
        System.out.println("SubClass : " + value);
    }
}

public class Polymorphism {
    static void print(SuperClass obj) {
        obj.output();
    }
    public static void main(String[] args) {
        SuperClass obj1 = new SuperClass(1);
        SubClass obj2 = new SubClass(1);
        print(obj1);
        print(obj2);
    }
}
```

SubClass



Exercise

Step 4, Make Queue Class to Implement Interface

- An Example of Interface, Slide # 4 - 9

Step 5, Make Package

- An Example of a Package, Slide # 14 - 15

Step 6, Generic Class Type(II)

- Refer to the slides as 10 - 11